

函数设计说明书

姓名: XXX
学号: XXXXXXXXXXXX
专业: 计算机科学与技术
指导老师: XXX

2025 年 6 月 1 日

目录

1 main.c 主控模块

1.1 read_handles — 读取用户列表

1.2 main — 程序主流程

2 cf_api.c 数据获取模块

2.1 write_callback — libcurl 写回调

2.2 cf_http_get — HTTP GET 请求

2.3 fetch_and_save — 请求+校验+缓存

2.4 rate_limiter 系列函数

3 json_utils.c 文件工具模块

3.1 read_text_file / write_text_file

3.2 load_cf_result_array / load_cf_result_first

4 analyzer.c 核心分析模块

4.1 rating_color / rating_title

4.2 process_contest_problems

4.3 count_ac_by_rating

4.4 build_user_summary

5 htmlgen.c HTML 生成模块

5.1 replace_placeholder

5.2 generate_html

5.3 generate_all_html

1.1 read_handles (in main.c)

功能说明

read_handles 函数负责从指定的文本文件中读取 Codeforces 用户名列表。文件格式为每行一个用户名，函数会自动去除每行首尾的空白字符（空格、制表符、回车、换行）。该函数在 main() 启动时被调用，用于获取需要处理的用户列表。

参数详解

path: 输入参数，指向用户列表文件的路径字符串。

handles[][MAX_HANDLE_LEN]: 输出参数，二维字符数组，用于存储读取到的用户名。MAX_HANDLE_LEN 定义为 64，即用户名最大长度。

返回值

int 类型。成功时返回读取到的用户数量；如果文件无法打开则返回 -1。

算法要点

使用 fgets

逐行读取，然后分别跳过前导空白字符和末尾空白字符。空行会被忽略。每个有效用户名通过 memmove 复制到输出数组中。最大支持 MAX_HANDLES (256) 个用户。

【代码实现】

```
static int read_handles(const char *path, char handles[][MAX_HANDLE_LEN]) {
    FILE *f = fopen(path, "r");
    if (!f) { fprintf(stderr, "Cannot open handle list: %s\n", path); return -1; }
    int count = 0;
    char buf[256];
    while (fgets(buf, sizeof(buf), f) && count < MAX_HANDLES) {
        char *s = buf;
        while (*s == ' ' || *s == '\t' || *s == '\r') s++; /* 跳过前导空白 */
        int len = (int)strlen(s);
        while (len > 0 && (s[len-1]=='\n' || s[len-1]=='\r' || s[len-1]==' ' || s[len-1]=='\t'))
            s[--len] = '\0'; /* 跳过末尾空白 */
        if (len > 0 && len < MAX_HANDLE_LEN) {
            memmove(handles[count], s, (size_t)len + 1);
            count++;
        }
    }
    fclose(f);
    return count;
}
```

1.2 main (in main.c)

功能说明

main 函数是整个程序的入口和流程编排中心。它按照以下流程组织整个系统的运行：读取用户列表 → 创建数据目录 → 初始化限速器 → 获取比赛列表 → 逐用户获取数据和分析 → 生成 HTML 报告。

参数详解

argc: 命令行参数个数。若不提供参数，默认使用 "sample_users.txt"。

argv: 参数字符串数组。argv[1] 可指定用户列表文件路径。

返回值

int 类型。正常执行完毕返回 0；如果没有任何有效用户则返回 1。

关键流程

- 1) 调用 read_handles() 读取用户列表。
- 2) 创建 data/ 和 output/ 目录。
- 3) 初始化 RateLimiter (2秒间隔) 。
- 4) 调用 cf_fetch_contest_list() 获取比赛列表（所有用户共享）。
- 5) 对每个用户依次执行：cf_fetch_user_info → cf_fetch_user_rating → cf_fetch_user_status。
- 6) 调用 build_user_summary() 进行综合分析。
- 7) 最后调用 generate_all_html() 生成所有 HTML 报告。

【代码实现】

```
int main(int argc, char *argv[]) {
    const char *handle_file = "sample_users.txt";
    if (argc > 1) handle_file = argv[1];
    printf("=== Codeforces Crawler 2025 ===\n");
    int handle_count = read_handles(handle_file, handles);
    if (handle_count <= 0) { fprintf(stderr, "No valid handles found\n"); return 1; }
    mkdir_p(DATA_DIR); mkdir_p(OUTPUT_DIR);
    RateLimiter rl; rate_limiter_init(&rl);
    cf_fetch_contest_list(DATA_DIR "/contest_list.json", &rl);
    cJSON *users_arr = cJSON_CreateArray();
    for (int i = 0; i < handle_count; i++) {
        const char *handle = handles[i];
        snprintf(path, sizeof(path), "%s/%s_user.json", DATA_DIR, handle);
        cf_fetch_user_info(handle, path, &rl); /* ■■■■■■ */
        snprintf(path, sizeof(path), "%s/%s_rating.json", DATA_DIR, handle);
        cf_fetch_user_rating(handle, path, &rl); /* ■■Rating■■ */
        snprintf(path, sizeof(path), "%s/%s_status.json", DATA_DIR, handle);
        cf_fetch_user_status(handle, path, &rl); /* ■■■■■■ */
        cJSON *summary = build_user_summary(handle, DATA_DIR, OUTPUT_DIR, 0);
        if (summary) cJSON_AddItemToArray(users_arr, summary);
    }
    generate_all_html(users_arr, TEMPLATE_PATH, OUTPUT_DIR);
    cJSON_Delete(users_arr);
    return 0;
}
```

2.1 write_callback (in cf_api.c)

功能说明

write_callback 是 libcurl 库的写回调函数。当 libcurl 通过 HTTP 协议接收到服务器返回的数据时，会自动调用此函数，将数据逐块写入动态分配的内存缓冲区中。这是 libcurl 标准的数据接收模式。

参数详解

contents：指向接收到的数据块的指针。

size：每个数据元素的大小（通常为 1）。

nmemb: 本次接收到的数据元素数量。

userp: 用户自定义指针, 指向 MemoryBlock 结构体。通过 curl_easy_setopt 的 CURLOPT_WRITEDATA 选项设置。

返回值

size_t 类型。返回实际处理的数据字节数, libcurl 据此确认数据已接收。返回 0 表示处理失败, libcurl 将终止请求。

算法要点

使用 realloc 动态扩展缓冲区。每次收到新数据时, 重新分配大小为 (现有大小 + 新数据大小 + 1) 的内存, 然后将新数据复制到原有数据末尾, 最后添加字符串结束符 \0。

【代码实现】

```
static size_t write_callback(void *contents, size_t size, size_t nmemb, void *userp) {
    size_t realsize = size * nmemb; /* ██████████ */
    struct MemoryBlock *mem = (struct MemoryBlock *)userp;
    char *ptr = realloc(mem->data, mem->size + realsize + 1); /* ██████ */
    if (!ptr) return 0; /* ██████ */
    mem->data = ptr;
    memcpy(&(mem->data[mem->size]), contents, realsize); /* ██████ */
    mem->size += realsize;
    mem->data[mem->size] = 0; /* ██████ */
    return realsize;
}
```

2.2 cf_http_get (in cf_api.c)

功能说明

cf_http_get 封装了 libcurl 的 HTTP GET 请求操作。创建一个 curl 句柄, 配置各项参数, 执行 HTTP 请求并返回完整的响应内容字符串。如果请求失败 (网络错误、超时等), 返回 NULL。

参数详解

url: 字符串指针, 指向完整的请求 URL, 例如

"https://codeforces.com/api/user.info?handles=tourist"。

返回值

char* 类型。成功时返回 malloc 分配的响应字符串, 调用者负责使用 free() 释放。失败 (如 DNS 解析失败、连接超时、HTTP 错误) 时返回 NULL。

关键配置

CURLOPT_TIMEOUT: 30 秒超时。

CURLOPT_FOLLOWLOCATION: 自动跟随 HTTP 重定向。

CURLOPT_SSL_VERIFYPEER/VERIFYHOST: Windows 环境下跳过 SSL 证书验证 (因缺少 CA 证书包)。

CURLOPT_USERAGENT: 设置 User-Agent 头标识。

【代码实现】

```
char *cf_http_get(const char *url) {
```

```

CURL *curl = curl_easy_init();
if (!curl) return NULL;
struct MemoryBlock chunk = {.data = malloc(1), .size = 0};
curl_easy_setopt(curl, CURLOPT_URL, url); /* ■■■■URL */
curl_easy_setopt(curl, CURLOPT_WRITEFUNCTION, write_callback); /* ■■■■ */
curl_easy_setopt(curl, CURLOPT_WRITEDATA, (void *)&chunk);
curl_easy_setopt(curl, CURLOPT_USERAGENT, "CF-Crawler/1.0");
curl_easy_setopt(curl, CURLOPT_TIMEOUT, 30L); /* 30■■■ */
curl_easy_setopt(curl, CURLOPT_FOLLOWLOCATION, 1L); /* ■■■■ */
curl_easy_setopt(curl, CURLOPT_SSL_VERIFYPEER, 0L); /* ■■SSL■■ */
curl_easy_setopt(curl, CURLOPT_SSL_VERIFYHOST, 0L);
CURLcode res = curl_easy_perform(curl); /* ■■■■ */
curl_easy_cleanup(curl);
if (res != CURLE_OK) {
    fprintf(stderr, "curl failed: %s\n", curl_easy_strerror(res));
    free(chunk.data); return NULL;
}
return chunk.data; /* ■■■■■■ */
}

```

2.3 fetch_and_save (in cf_api.c)

功能说明

fetch_and_save 是通用的"请求 → 校验 → 缓存"流程封装。它将 API 调用、响应校验和文件缓存三个步骤组合为一个函数，确保每次 API 调用都经过限速器控制，并验证响应格式正确后再缓存。

参数详解

url: API 请求的完整 URL。

path: 本地缓存文件的保存路径，例如 "data/tourist_user.json"。

rl: 指向 RateLimiter 结构体的指针，用于控制请求频率。

返回值

int 类型。成功返回 0；如果网络请求失败、JSON 校验不通过或文件写入失败，返回 -1。

校验逻辑

使用 cJSON 解析响应 JSON，检查 status 字段是否为 "OK"。如果 status 不是 "OK"（例如用户不存在时 API 返回 "FAILED"），则打印错误信息并返回 -1。同时会输出 API 返回的 comment 字段（包含具体错误原因，如 "handle: User not found"）。

【代码实现】

```

int fetch_and_save(const char *url, const char *path, RateLimiter *rl) {
    rate_limiter_wait(rl); /* ■■■■ */
    printf(" Fetching: %s\n", url);
    char *response = cf_http_get(url); /* HTTP GET■■■ */
    if (!response) return -1;
    rate_limiter_tick(rl); /* ■■■■ */
    cJSON *root = cJSON_Parse(response); /* ■■JSON */
    if (!root) { free(response); return -1; }
    cJSON *status = cJSON_GetObjectItem(root, "status");
    if (!status || strcmp(status->valuestring, "OK") != 0) {

```

```

cJSON *comment = cJSON_GetObjectItem(root, "comment");
fprintf(stderr, "CF API Error: %s\n",
comment ? comment->valuelstring : status->valuelstring);
cJSON_Delete(root); free(response); return -1;
}
cJSON_Delete(root);
int ret = write_text_file(path, response); /* ■■■■ */
free(response);
return ret;
}

```

2.4 rate_limiter 系列函数 (in rate_limiter.c)

功能说明

rate_limiter 模块实现了跨平台的 API 请求频率限制功能。Codeforces 官方 API 限制为每秒最多 1 次请求，本模块采用更保守的 2 秒间隔策略。包含三个函数：init（初始化）、wait（等待）、tick（标记）。时间获取使用高精度时钟：Windows 上使用 GetSystemTimeAsFileTime（100 纳秒精度），POSIX 系统上使用 clock_gettime(CLOCK_MONOTONIC)。

数据结构

RateLimiter 结构体包含两个成员：

last_request_time: double 类型，记录上一次 API 请求的时间戳（单位：秒）。

min_interval: double 类型，最小请求间隔（固定为 2.0 秒）。

参数详解

rl: 指向 RateLimiter 结构体的指针。所有函数均通过该指针操作限速器状态。

返回值

所有函数均为 void 类型，无返回值。

算法要点

wait() 函数首先检查 last_request_time 是否已初始化（为 0 表示首次请求，无需等待）。然后计算当前时间与上次请求的时间差 elapsed，若 elapsed < min_interval，则休眠 (min_interval - elapsed) 秒。休眠使用平台相关的 Sleep/nanosleep 函数。

【代码实现】

```

/* Windows: ■■ GetSystemTimeAsFileTime ■■■■■■ */
static double get_wall_time(void) {
    FILETIME ft;
    GetSystemTimeAsFileTime(&ft); /* ■■100ns■■■■ */
    unsigned long long t = ft.dwHighDateTime;
    t <<= 32;
    t |= ft.dwLowDateTime;
    return (double)t / 10000000.0; /* ■■■■ */
}

void rate_limiter_init(RateLimiter *rl) {
    rl->last_request_time = 0.0;
    rl->min_interval = 2.0; /* ■■2■■■ */
}

void rate_limiter_wait(RateLimiter *rl) {
    if (rl->last_request_time == 0.0) return; /* ■■■■■■■■ */
}

```

```
double now = get_wall_time();
double elapsed = now - rl->last_request_time;
if (elapsed < rl->min_interval) {
    double sleep_sec = rl->min_interval - elapsed;
    Sleep((DWORD)(sleep_sec * 1000)); /* ■■■■■■ */
}
}

void rate_limiter_tick(RateLimiter *rl) {
    rl->last_request_time = get_wall_time(); /* ■■■■■■ */
}
```

3.1 read_text_file / write_text_file (in json_utils.c)

功能说明

这对函数提供了最底层的文本文件读写能力。read_text_file 将整个文件读入一个 malloc 分配的字符串缓冲区，write_text_file 将字符串完整写入文件。它们是所有数据持久化的基础。

参数详解

read_text_file: path — 要读取的文件路径。

write_text_file: path — 要写入的文件路径；content — 待写入的字符串内容。

返回值

read_text_file: 成功返回 char* 指针（包含文件全部内容），文件不存在或读取失败返回 NULL。

write_text_file: 成功返回 0，写入字节数不匹配时返回 -1。

注意事项

文件以二进制模式 ("rb"/"wb") 打开，保证跨平台兼容性（Windows 下不会将 \r\n 转换为 \n）。read_text_file 使用 malloc 分配内存，调用者必须在不再使用时 free() 释放。

【代码实现】

```
char *read_text_file(const char *path) {
    FILE *f = fopen(path, "rb"); /* ■■■■■■ */
    if (!f) return NULL;
    fseek(f, 0, SEEK_END); long len = ftell(f); /* ■■■■■■ */
    rewind(f);
    char *content = (char *)malloc((size_t)len + 1); /* ■■■■■■ */
    if (!content) { fclose(f); return NULL; }
    size_t r = fread(content, 1, (size_t)len, f); /* ■■■■■■ */
    content[r] = '\0';
    fclose(f); return content;
}

int write_text_file(const char *path, const char *content) {
    FILE *f = fopen(path, "wb"); /* ■■■■■■ */
    if (!f) return -1;
    size_t len = strlen(content);
    size_t w = fwrite(content, 1, len, f); /* ■■■■■■ */
    fclose(f);
    return (w == len) ? 0 : -1;
}
```


3.2 load_cf_result_array / load_cf_result_first (in json_utils.c)

功能说明

这两个函数用于从 CF 标准 API 响应文件中提取数据。CF API

返回格式为: {"status":"OK","result":[...]}. load_cf_result_array 提取整个 result

数组, load_cf_result_first 提取数组的第一个元素 (适用于 user.info 等单元素数组)。返回的 cJSON 对象是深拷贝 (cJSON_Duplicate), 与原始解析树独立, 避免了内存管理冲突。

参数详解

path: 缓存 JSON 文件的路径。例如 "data/tourist_user.json"。

返回值

cJSON* 类型。成功返回提取的数组或单元素指针, 调用者必须使用 cJSON_Delete() 释放。如果文件不存在、status 不为 "OK" 或 result 不是数组, 返回 NULL。

设计考量

使用深拷贝 (cJSON_Duplicate 的第二个参数为 1) 的原因: 原始解析树在函数返回前会被 cJSON_Delete 销毁, 浅拷贝的指针会成为悬空指针。深拷贝确保了返回数据的独立性。

【代码实现】

```
cJSON *load_cf_result_array(const char *path) {
    cJSON *root = read_json_file(path); /* ■■■■■■JSON */
    if (!root) return NULL;
    cJSON *s = cJSON_GetObjectItem(root, "status");
    if (!s || strcmp(s->valuestring, "OK") != 0) /* ■■status */
    { cJSON_Delete(root); return NULL; }
    cJSON *r = cJSON_GetObjectItem(root, "result");
    if (!r || !cJSON_IsArray(r)) /* ■■result■■ */
    { cJSON_Delete(root); return NULL; }
    cJSON *ret = cJSON_Duplicate(r, 1); /* ■■■ */
    cJSON_Delete(root); return ret;
}

cJSON *load_cf_result_first(const char *path) {
    cJSON *arr = load_cf_result_array(path);
    if (!arr) return NULL;
    cJSON *f = cJSON_GetArrayItem(arr, 0); /* ■■■■■■ */
    cJSON *ret = f ? cJSON_Duplicate(f, 1) : NULL; /* ■■■■ */
    cJSON_Delete(arr); return ret;
}
```

4.1 rating_color / rating_title (in analyzer.c)

功能说明

这两个函数实现了 Codeforces 官方定义的 10 级颜色体系。rating_color

根据等级分返回对应的十六进制颜色字符串, 用于在 HTML 报告中按 CF 规范着色显示用户信息。rating_title

根据等级分返回对应的头衔名称。整个系统的所有颜色显示均依赖此映射。

参数详解

rating: 整数, 表示用户当前的 Codeforces 等级分。范围从 0 到 4000+。

返回值

rating_color 返回颜色字符串（如 "#FF0000"）；rating_title 返回头衔字符串（如 "Grandmaster"）。返回值均为静态常量字符串指针，无需释放。

颜色体系

3000+: 黑色 #000000 (Legendary Grandmaster)
2600-2999: 深红 #CC0000 (International Grandmaster)
2400-2599: 红色 #FF0000 (Grandmaster)
2300-2399: 橙色 #FF8C00 (International Master)
2100-2299: 橙色 #FF8C00 (Master)
1900-2099: 紫色 #AA00AA (Candidate Master)
1600-1899: 蓝色 #0000FF (Expert)
1400-1599: 青色 #03A89E (Specialist)
1200-1399: 绿色 #008000 (Pupil)
0-1199: 灰色 #808080 (Newbie)

【代码实现】

```
const char *rating_color(int rating) {
    if (rating >= 3000) return "#000000"; /* Legendary Grandmaster */
    if (rating >= 2600) return "#CC0000"; /* International Grandmaster */
    if (rating >= 2400) return "#FF0000"; /* Grandmaster */
    if (rating >= 2300) return "#FF8C00"; /* International Master */
    if (rating >= 2100) return "#FF8C00"; /* Master */
    if (rating >= 1900) return "#AA00AA"; /* Candidate Master */
    if (rating >= 1600) return "#0000FF"; /* Expert */
    if (rating >= 1400) return "#03A89E"; /* Specialist */
    if (rating >= 1200) return "#008000"; /* Pupil */
    return "#808080"; /* Newbie */
}

const char *rating_title(int rating) {
    if (rating >= 3000) return "Legendary Grandmaster";
    if (rating >= 2600) return "International Grandmaster";
    if (rating >= 2400) return "Grandmaster";
    if (rating >= 2300) return "International Master";
    if (rating >= 2100) return "Master";
    if (rating >= 1900) return "Candidate Master";
    if (rating >= 1600) return "Expert";
    if (rating >= 1400) return "Specialist";
    if (rating >= 1200) return "Pupil";
    return "Newbie";
}
```

4.2 process_contest_problems (in analyzer.c)

功能说明

process_contest_problems 是赛后补题检测算法的核心实现。它遍历所有提交记录中属于指定比赛的条目，通过比较每次提交的创建时间与比赛结束时间，精确判断每道题的三种状态：赛时通过（AC）、赛时未通过、赛后补题通过。

算法原理

比赛结束时间的计算：从 contest_list 中获取每场比赛的 startTimeSeconds 和 durationSeconds，比赛结束时间 = startTimeSeconds + durationSeconds。

判定逻辑：对于每道题的所有提交，如果存在任意一条状态为 "OK" 且 creationTimeSeconds ≤ 比赛结束时间的提交，则该题标记为"赛时 AC"。如果只有比赛结束时间之后的 AC 提交，则标记为"赛后补题"。如果没有任何 AC 但有错误提交，则标记为"赛时未通过"。

参数详解

status: cJSON 数组，user.status 接口返回的完整提交列表。

contest_id: 整数，目标比赛的 ID 号。

contest_end: 比赛结束的 Unix 时间戳（秒）。

probs: ProblemState 数组，用于输出每道题的分析结果。

max_probs: 数组容量上限（定义为 26，对应 A-Z）。

返回值

int 类型，返回该比赛中识别到的不同题目数量。

【代码实现】

```
static int process_contest_problems(cJSON *status, int contest_id,
long long contest_end, ProblemState *probs, int max_probs) {
    int prob_count = 0;
    if (!status) return 0;
    cJSON *sub;
    cJSON_ArrayForEach(sub, status) { /* ████████ */
        cJSON *scid = cJSON_GetObjectItem(sub, "contestId");
        if (!scid || scid->valueint != contest_id) continue; /* ████████ */
        cJSON *sproblem = cJSON_GetObjectItem(sub, "problem");
        cJSON *sindex = cJSON_GetObjectItem(sproblem, "index");
        /* ... ██████████ ProblemState ███ ... */
        cJSON *sverdict = cJSON_GetObjectItem(sub, "verdict");
        cJSON *sctime = cJSON_GetObjectItem(sub, "creationTimeSeconds");
        long long sub_time = (long long)sctime->valuedouble;
        if (strcmp(sverdict->valuestring, "OK") == 0) {
            if (sub_time > 0 && sub_time <= contest_end)
                probs[idx].in_contest_ac = 1; /* ██████ */
            else
                probs[idx].post_ac = 1; /* ██████ */
        } else {
            probs[idx].rejected++; /* ██████ */
        }
    }
    qsort(probs, prob_count, sizeof(ProblemState), prob_cmp); /* ██████ */
    return prob_count;
}
```

4.3 count_ac_by_rating (in analyzer.c)

功能说明

count_ac_by_rating

统计用户在指定时间窗口内通过的题目按照题目难度（rating）的分布情况。遍历所有 AC

提交，提取题目的 rating 值，按 rating 分组计数。该函数在 build_user_summary 中被调用 4 次，分别生成全部/一年/180天/一个月 的直方图数据。

参数详解

status: user.status 接口返回的完整提交列表 (cJSON 数组)。

hist: 输出参数，cJSON 对象，用于存储 {rating值: 计数} 的键值对。

now_ts: 当前 Unix 时间戳，用于计算时间窗口的起始点。

window_sec: 时间窗口长度 (秒)。0 表示全部历史，不进行时间过滤。

返回值

void，无返回值。结果直接写入 hist 参数中。

实现细节

使用 cJSON 对象作为哈希表。rating 值作为字符串 key (如 "800"、"1500"、"2400")，计数作为 value。这种实现无需预分配数组，可处理任意分布的 rating 值。

【代码实现】

```
static void count_ac_by_rating(cJSON *status, cJSON *hist,
long long now_ts, long long window_sec) {
    if (!status) return;
    cJSON *sub;
    cJSON_ArrayForEach(sub, status) {
        cJSON *verdict = cJSON_GetObjectItem(sub, "verdict");
        if (!verdict || strcmp(verdict->valuestring, "OK") != 0) continue; /* AC */
        cJSON *ctime = cJSON_GetObjectItem(sub, "creationTimeSeconds");
        if (!ctime) continue;
        long long ts = (long long)ctime->valuedouble;
        if (window_sec > 0 && now_ts - ts > window_sec) continue; /* 窗口 */
        cJSON *problem = cJSON_GetObjectItem(sub, "problem");
        cJSON *prating = cJSON_GetObjectItem(problem, "rating");
        int r = prating ? prating->valueint : 0;
        char key[16];
        snprintf(key, sizeof(key), "%d", r); /* rating */
        cJSON *existing = cJSON_GetObjectItem(hist, key);
        if (existing)
            existing->valueint++; /* 计数 */
        else
            cJSON_AddNumberToObject(hist, key, 1); /* 初始化 */
    }
}
```

4.4 build_user_summary (in analyzer.c)

功能说明

build_user_summary 是分析模块的核心入口函数，也是整个系统中最复杂的函数。它整合了 4 个数据源（用户信息、rating

历史、提交记录、比赛列表），完成以下分析任务：基本信息提取、比赛记录汇总、赛后补题检测、AC 难度直方图统计、180 天活跃度统计。最终返回一个完整的 cJSON 结果树。

处理流程

- 1) 加载 4 个缓存文件: user_info、rating_history、status、contest_list。
- 2) 提取基本信息: handle、rating、maxRating、rank、avatar 等。
- 3) 调用 adjust_rating_rank() 修正 API 可能返回的错误头衔。
- 4) 遍历 rating_history 数组, 对每场比赛: 获取起止时间, 调用 process_contest_problems() 分析各题状态。
- 5) 统计 180 天内的参赛次数和最高 rating。
- 6) 调用 count_ac_by_rating() 4 次, 生成不同时间窗口的直方图。
- 7) 释放所有临时数据, 返回结果树。

返回值

cJSON* 类型。包含完整的分析结果树, 结构如下:

```
{handle, rating, maxRating, rank, maxRank, titlePhoto, avatar,
contestCount, contestCount180, maxRating180,
contests: [{contestId, contestName, timestamp, oldRating, newRating,
rank, solvedCount, problems: [...], upsolved: [...]}],
histAll: {...}, histYear: {...}, hist180: {...}, histMonth: {...}}
```

【代码实现】

```
cJSON *build_user_summary(const char *handle, const char *data_dir,
const char *output_dir, int max_standings) {
cJSON *result = cJSON_CreateObject();
if (!result) return NULL;
/* 1. 加载缓存文件 */
cJSON *user_info = load_cf_result_first(user_path);
cJSON *rating_history = load_cf_result_array(rating_path);
cJSON *status = load_cf_result_array(status_path);
cJSON *contest_list = load_cf_result_array(clist_path);
/* 2. 修正 API 返回的错误头衔 */
adjust_rating_rank(user_info);
/* 3. 统计参赛次数和最高 rating */
cJSON_AddStringToObject(result, "handle", handle);
cJSON_AddNumberToObject(result, "rating", rating);
cJSON_AddStringToObject(result, "rank", rating_title(rating));
/* 4. 遍历比赛, 分析各题状态 */
cJSON *contests_arr = cJSON_AddArrayToObject(result, "contests");
cJSON_ArrayForEach(item, rating_history) {
/* 调用 process_contest_problems */
}
/* 5. 统计 180 天内的参赛次数和最高 rating */
cJSON_AddNumberToObject(result, "contestCount180", contest_count_180);
cJSON_AddNumberToObject(result, "maxRating180", max_rating_180);
/* 6. 生成不同时间窗口的直方图 */
count_ac_by_rating(status, hist_all, now_ts, 0); /* 全部 */
count_ac_by_rating(status, hist_year, now_ts, 365*86400); /* 一年 */
count_ac_by_rating(status, hist_180, now_ts, 180*86400); /* 180 天 */
count_ac_by_rating(status, hist_month, now_ts, 30*86400); /* 一个月 */
/* 7. 释放临时数据 */
cJSON_Delete(user_info); cJSON_Delete(rating_history);
if (status) cJSON_Delete(status);
if (contest_list) cJSON_Delete(contest_list);
return result;
```

```
}
```

5.1 replace_placeholder (in htmlgen.c)

功能说明

replace_placeholder 是一个简单的模板替换引擎。它扫描字符串中所有出现的占位符（如 {{TITLE}}），将其替换为目标值，并返回新分配的字符串。该函数是 HTML 生成的基础，负责将数据注入 HTML 模板。

参数详解

tmpl: 模板字符串，包含占位符，例如从 template.html 读取的内容。

placeholder: 目标占位符，格式为 "{{NAME}}", 如 "{{TITLE}}", "{{DATA_JSON}}".

value: 替换值字符串，将替换所有出现的占位符。

返回值

char* 类型。返回新分配的字符串，调用者负责使用 free() 释放。如果模板中不含指定占位符，返回原模板的 strdup 副本。

算法要点

先统计占位符出现次数以计算输出缓冲区大小（避免缓冲区溢出），然后遍历模板字符串，逐段复制原内容，遇到占位符时插入替换值。

【代码实现】

```
static char *replace_placeholder(const char *tmpl,
const char *placeholder, const char *value) {
    size_t tlen = strlen(tmpl), plen = strlen(placeholder);
    size_t vlen = strlen(value);
    /* 1. 统计占位符出现次数 */
    int count = 0;
    const char *p = tmpl;
    while ((p = strstr(p, placeholder)) != NULL) { count++; p += plen; }
    if (count == 0) return strdup(tmpl);
    /* 2. 计算输出缓冲区大小 */
    size_t out_len = tlen + count * (vlen - plen) + 1;
    char *out = (char *)malloc(out_len);
    if (!out) return NULL;
    /* 3. 复制内容并替换 */
    char *dst = out;
    const char *src = tmpl;
    const char *next;
    while ((next = strstr(src, placeholder)) != NULL) {
        size_t seg_len = next - src;
        memcpy(dst, src, seg_len); dst += seg_len; /* 复制原内容 */
        memcpy(dst, value, vlen); dst += vlen; /* 复制替换值 */
        src = next + plen; /* 跳过占位符 */
    }
    strcpy(dst, src); /* 复制剩余内容 */
    return out;
}
```

5.2 generate_html (in htmlgen.c)

功能说明

generate_html 函数执行"模板 → 替换 → 输出"的完整 HTML 生成流程。它读取模板文件，将 cJSON 数据序列化为 JSON 字符串，替换模板中的 {{TITLE}} 和 {{DATA_JSON}}

占位符，最终将结果写入指定的输出文件。生成的 HTML

文件自包含所有数据，无需额外文件即可在浏览器中打开。

参数详解

template_path: HTML 模板文件的路径，通常为 "web/template.html"。

output_path: 输出 HTML 文件的路径。

title: 页面标题，替换 {{TITLE}} 占位符，例如 "Codeforces - tourist"。

data: cJSON 数据对象或数组，序列化为 JSON 后替换 {{DATA_JSON}} 占位符。

返回值

int 类型。成功返回 0。如果模板文件无法读取、JSON 序列化失败或文件写入失败，返回 -1。

数据嵌入方式

HTML 模板中通过 const CF_DATA = {{DATA_JSON}}; 的方式将数据嵌入 JavaScript 变量。浏览器加载页面时，JavaScript 代码根据 CF_DATA

的类型（数组或对象）自动选择渲染模式：数组 → 多用户首页；对象 → 单用户详情页。

【代码实现】

```
int generate_html(const char *template_path, const char *output_path,
const char *title, cJSON *data) {
    /* ■■■1■■■■■■■■■■ */
    char *tmpl = read_text_file(template_path);
    if (!tmpl) { fprintf(stderr, "Cannot read template\n"); return -1; }
    /* ■■■2■■■■■■■■■■ cJSON ■■■ */
    char *json_str = data ? cJSON_PrintUnformatted(data) : strdup("null");
    if (!json_str) { free(tmpl); return -1; }
    /* ■■■3■■■■{{TITLE}}■■■■ */
    char *after_title = replace_placeholder(tmpl, "{{TITLE}}", title);
    free(tmpl);
    if (!after_title) { free(json_str); return -1; }
    /* ■■■4■■■■{{DATA_JSON}}■■■■ */
    char *result = replace_placeholder(after_title, "{{DATA_JSON}}", json_str);
    free(after_title); free(json_str);
    if (!result) return -1;
    /* ■■■5■■■■■■■■■■ */
    int ret = write_text_file(output_path, result);
    free(result);
    printf(" Generated: %s\n", output_path);
    return ret;
}
```

5.3 generate_all_html (in htmlgen.c)

功能说明

generate_all_html 负责批量生成完整的 HTML 报告集合。首先生成多用户首页 index.html，然后遍历用户数组，为每个用户生成独立的详情页 report_.html。生成的 index.html 中包含所有用户的概要信息（列表），并通过搜索框支持实时过滤。每个 report_.html 则是带有 ECharts 图表的完整分析报告。

参数详解

users: cJSON 数组，包含所有用户的完整分析数据（由 build_user_summary 生成）。

template_path: HTML 模板文件的路径。

output_dir: 输出目录路径，生成的 HTML 文件将放置在此目录下。

返回值

int 类型。成功返回 0；如果 users 为 NULL 返回 -1。

输出文件结构

output/index.html — 多用户首页，显示所有用户的 rating、头衔、比赛次数等概要信息。

output/report_.html — 每个用户独立的详情页，包含 ECharts 折线图、柱状图、比赛记录表。

【代码实现】

```
int generate_all_html(cJSON *users, const char *template_path,
const char *output_dir) {
    if (!users) return -1;
    /* 1. ■■■■■■■■ index.html */
    char index_path[512];
    snprintf(index_path, sizeof(index_path), "%s/index.html", output_dir);
    generate_html(template_path, index_path,
"Codeforces Analytics - Users", users);
    /* 2. ■■■■■■■■■■■■■■■■■■■■■■ */
    cJSON *user;
    cJSON_ArrayForEach(user, users) {
        cJSON *handle = cJSON_GetObjectItem(user, "handle");
        if (!handle || !handle->valuestring) continue;
        char detail_path[512], page_title[256];
        snprintf(detail_path, sizeof(detail_path),
"%s/report_%s.html", output_dir, handle->valuestring);
        snprintf(page_title, sizeof(page_title),
"Codeforces - %s", handle->valuestring);
        generate_html(template_path, detail_path, page_title, user);
    }
    return 0;
}
```